

useRef

Escape hatch for mutable values and DOM references that persist across renders without re-rendering.

Basic API

```
const ref = useRef(initialValue);
```

DOM manipulation

Access DOM nodes for imperative operations.

```
function SearchInput() {
  const inputRef = useRef(null);
  return (
    <>
      <input ref={inputRef} />
      <button onClick={() => inputRef.current.focus()}
    >Focus</button>
    </>
  );
}
```

Mutable values

Store values that persist across renders without triggering re-renders.

```
function SearchBox() {
  const searchCount = useRef(0);
  const handleSearch = () => {
    searchCount.current += 1;
    console.log(`Search performed ${searchCount.current}
times`);
  };
  return <button onClick={handleSearch}>Search</button>;
}
```

- ✓ **No re-renders** Changing `ref.current` doesn't trigger updates
- ✓ **Imperative APIs** Focus, scroll, play/pause, measurements
- ✓ **Instance variables** Store timers, subscriptions, previous values

Custom Hooks

Extract and reuse stateful logic between components with functions starting with "use".

State logic

Each component gets its own independent state.

```
function useCounter(initialValue = 0) {
  const [count, setCount] = useState(initialValue);
  const increment = () => setCount(c => c + 1);
  return { count, increment };
}
```

- ✓ **Start with "use"**
Naming convention enables linter rules
- ✓ **Composition**
Build complex Hooks from simpler Hooks

Rules of Hooks

Follow these rules to ensure Hooks work correctly and avoid bugs.

Only call Hooks at the top level

Always use Hooks at the top level of your React function, before any early returns.

```
function Profile({ user }) {
  if (user.isLoggedIn) {
    const [status, setStatus] = useState('online'); // ✗
  }
  // Inside condition
  for (let i = 0; i < 5; i++) {
    const ref = useRef(null); // ✗ Inside loop
  }
}
```

Only call Hooks from React functions

Call Hooks from React function components and custom Hooks only.

```
function calculateTotal() {
  const [total, setTotal] = useState(0); // ✗ Regular
  function
}
```

- ✓ **Linter helps** `'eslint-plugin-react-hooks'`
catches rule violations automatically
- ✓ **Order matters** React relies on call order to track state
- ✓ **Function components**
Only works in function components and custom Hooks



certificates.dev



React Certification

Learn more at certificates.dev/react

